

0.75.2. Arrivé en (0, 0), le A devient B — doublé, deux positions

Deux A, deux départs, la même case d'arrivée : chacun devient B en arrivant en (0, 0).

1. Ce qu'il faut comprendre

C'est le 0.75 et le 0.75.1 réunis : deux A, l'un parti de (10, 0), l'autre de (10, 8). La même croix les déplace ensemble.

Chacun a ses variables : `x`, `y`, `lettre` pour le premier ; `xb`, `yb`, `lettreB` pour le second. Et chacun a **sa condition**, sur **sa** position, mais vers la **même** case : (0, 0).

Ils n'y arrivent pas en même temps. Avec la flèche gauche, le premier arrive en (0, 0) et devient B ; le second, en (0, 8), reste un A. Avec la flèche haut, le premier ne peut plus monter (il est déjà en haut) ; le second monte, arrive en (0, 0), et devient B à son tour.

À la fin, les deux B sont sur la même case, (0, 0) : on n'en voit qu'un. Chaque lettre ne connaît que sa propre position ; rien ne les empêche de se retrouver au même endroit.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_croix>` → [0.66.3. La fonction `deplace\_croix\(\)` — suivre la croix, case par case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction `nombre\(\)` — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1328 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      86 o ← la console, à l'allumage - la mise en route de la console
deplace_croix() 217 o ← main() - une fonction fournie par le langage
main()          133 o ← démarrage - ta fonction
VBlank          8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase     10 o ← effacer(), poser() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
LireManette     37 o ← bouton() - lit les huit boutons
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

nombre()	× 4	40 o sur place, dans main()	+ EcrireNombre (33 o)
bouton()	× 4	20 o sur place, dans deplace_croix()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans deplace_croix()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans deplace_croix()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o sur place, dans deplace_croix()	

Tes fonctions appelées :

deplace_croix() × 2 42 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
#include <nombre>           // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 10;           // le premier A : départ (10, 0)
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;
```

4. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;
```

5. UNE VARIABLE — XB

```
uint8_t xb = 10;          // le second A : départ (10, 8)
```

6. UNE VARIABLE — YB

```
uint8_t yb = 8;
```

7. UNE VARIABLE — LETTREB

```
uint8_t lettreB = 0;
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[lettre], 250);    // le premier
        deplace_croix(xb, yb, ALPHABET[lettreB], 250); // le second, dans la même image
        nombre(0, 17, x);                             // la position du premier
        nombre(4, 17, y);
        nombre(10, 17, xb);                            // celle du second
        nombre(14, 17, yb);

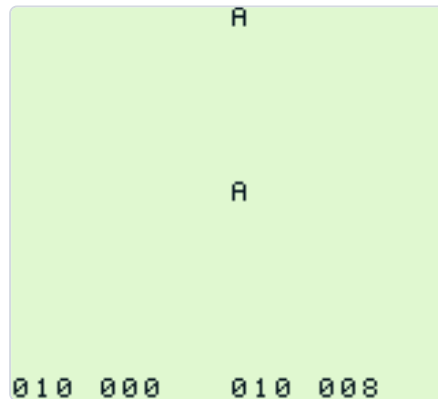
        // La même case pour les deux : (0, 0). Chacun regarde SA position.
        // 1. Le premier devient B quand il arrive en (0, 0).
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
        // 2. Le second aussi, quand il y arrive à son tour.
        // Sur une ligne : if (xb == 0 && yb == 0) lettreB = 1;
        // Avec des accolades, la même chose :
        if (xb == 0 && yb == 0) {
            lettreB = 1;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux A ; à gauche, le premier arrive en (0, 0) et devient B ; en montant, le second y arrive aussi et devient B.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à gauche : le premier est B, le second encore A

✓ en montant, le second devient B en (0, 0)

✓ un B en (0, 0)

Ensuite : **0.76. Un déclencheur en (0, 0) : actif** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++